

HTML, CSS & Web Fundamentals – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 14 Sep 2026 – 15 Sep 2026 (Days 29–30)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Phase 0 — Read BEFORE React (Days 29–30 or earlier).

Prerequisites: None.

Unlocks: React (03), Next.js (04).

Why this exists (don't skip)

React is **not** a replacement for HTML/CSS — it generates HTML. At 80 LPA interviews you will be asked:

- How does the browser render a page?
- What is the CSS box model?
- Why does CORS exist?
- Difference between `<div>` and semantic HTML?

Skipping this causes confusion with JSX, styling, SSR hydration, and frontend system design.

1. How the Web Works

User types URL

- DNS resolves domain to IP
- TCP connection to server
- HTTP request (GET /index.html)
- Server responds (200 OK + HTML)
- Browser parses HTML → DOM tree
- Browser parses CSS → CSSOM tree
- Render tree → Layout → Paint → Composite
- JavaScript downloads and executes

Term	Meaning
DNS	Phone book: <code>google.com</code> → <code>142.250.x.x</code>
TCP	Reliable delivery of data packets
HTTP	Request/response protocol on top of TCP
DOM	Tree representation of HTML in memory
CSSOM	Tree representation of CSS

2. HTML Essentials

Document structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My App</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>...</header>
  <main>...</main>
  <footer>...</footer>
  <script src="app.js"></script>
</body>
</html>
```

Semantic tags (use these, not div soup)

Tag	Use for
<header>	Page/section header
<nav>	Navigation links
<main>	Primary content (one per page)
<article>	Self-contained content
<section>	Thematic grouping
<aside>	Sidebar, related content
<footer>	Footer
<figure> / <figcaption>	Images with captions

Forms (React forms build on this)

```
<form action="/login" method="POST">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>
  <label for="password">Password</label>
  <input type="password" id="password" name="password" minlength="8">
  <button type="submit">Login</button>
</form>
```

Important attributes

- `id` — unique identifier (CSS `#id` , JS `getElementById`)
- `class` — reusable styling (CSS `.class`)
- `data-*` — custom data attributes
- `aria-*` — accessibility

3. CSS Essentials

Box model



```
* { box-sizing: border-box; } /* always use this */

.card {
  width: 300px;
  padding: 16px;
  margin: 8px;
  border: 1px solid #ccc;
  border-radius: 8px;
}
```

Flexbox (most used layout)

```
.container {
  display: flex;
  justify-content: center; /* horizontal */
  align-items: center;     /* vertical */
  gap: 16px;
  flex-wrap: wrap;
}

.item {
  flex: 1; /* grow to fill space */
}
```

CSS Grid

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 24px;
}
```

Responsive design

```
/* Mobile first */  
.sidebar { display: none; }  
  
@media (min-width: 768px) {  
  .sidebar { display: block; width: 250px; }  
}
```

CSS variables (used in theming)

```
:root {  
  --color-primary: #2563eb;  
  --spacing-md: 16px;  
}  
  
.button {  
  background: var(--color-primary);  
  padding: var(--spacing-md);  
}
```

4. DOM & JavaScript in Browser

```
// Select elements
document.getElementById("email");
document.querySelector(".card");
document.querySelectorAll("li");

// Modify
element.textContent = "Hello";
element.classList.add("active");
element.style.color = "red";

// Events
button.addEventListener("click", (e) => {
  e.preventDefault();
  console.log("clicked");
});

// Create elements
const div = document.createElement("div");
div.textContent = "New";
parent.appendChild(div);
```

React replaces manual DOM manipulation — but interviewers expect you to know what React abstracts.

5. Browser Storage

API	Scope	Size	Use
localStorage	Permanent	~5MB	Theme preference
sessionStorage	Tab session	~5MB	Form draft
cookies	Sent with HTTP	~4KB	Auth tokens (httpOnly preferred)
IndexedDB	Permanent	Large	Offline apps

6. Connection to React/Next.js

Web fundamental	React equivalent
<code>class="btn"</code>	<code>className="btn"</code>
<code>for="email"</code>	<code>htmlFor="email"</code>
<code>onclick</code>	<code>onClick</code>
Manual DOM updates	Virtual DOM + reconciliation
<code><a href></code>	<code><Link href></code> (Next.js)
CSS file	CSS Modules / Tailwind / styled-components

7. Exercises

1. Build static resume page (HTML only) — semantic tags
2. Style with Flexbox — header, sidebar, main layout
3. Add dark mode with CSS variables
4. Form with validation (vanilla JS, no framework)
5. Recreate exercise 4 in React — compare code

Where to practice: freeCodeCamp Responsive Web Design, MDN HTML/CSS tutorials

8. Interview Q&A (with answers)

Q: What happens when you type `google.com` in the browser?

A: DNS lookup resolves the domain to an IP address. Browser opens TCP connection (TLS handshake for HTTPS). Browser sends HTTP GET request. Server responds with HTML. Browser parses HTML into DOM, fetches CSS/JS resources in parallel. CSSOM + DOM → render tree → layout (calculate positions) → paint pixels → JavaScript executes and may modify DOM. Subsequent navigations may use HTTP cache or service worker.

Q: Explain the CSS box model.

A: Every element is a rectangular box with content, padding, border, and margin layers. `box-sizing: content-box` (default) adds padding/border outside width. `border-box` includes padding

and border in width — preferred for predictable layouts. Margin collapses vertically between adjacent block elements.

Q: Flexbox vs Grid — when to use which?

A: Flexbox is one-dimensional (row OR column) — nav bars, centering, equal-height cards in a row. Grid is two-dimensional (rows AND columns) — page layouts, dashboards, complex grids. Often combined: Grid for page shell, Flexbox for components inside cells.

9. Gate to Phase 3 (React)

Before opening React PDF, you must:

- ☐ Build one static page with semantic HTML
 - ☐ Use Flexbox for layout
 - ☐ Explain box model aloud
 - ☐ Know what DOM is vs Virtual DOM (preview)
-

Product Company Interview Bank – HTML and CSS Web Fundamentals (50 Questions)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Target companies: Google, Meta, Microsoft, Amazon, Adobe, Atlassian, Uber, Flipkart, Salesforce, Stripe **Rule:** Full terms only — no unexplained abbreviations. Write for absolute beginners who know nothing.

Question 1: What is HyperText Markup Language, and what role does it play in a web page?

Answer: HyperText Markup Language is the standard language used to describe the structure and meaning of content on a web page. It uses tags such as headings, paragraphs, links, and images to tell the browser what each piece of content is. HyperText Markup Language does not control visual styling—that is handled by Cascading Style Sheets. Together, structure from HyperText Markup Language and presentation from Cascading Style Sheets create the pages users see in the browser.

Question 2: What is the difference between a block-level element and an inline element?

Answer: A block-level element starts on a new line and stretches to fill the full available width of its container, stacking vertically like boxes in a column. Examples include `div`, `p`, and `section`. An inline element flows within a line of text and only takes up as much width as its content needs. Examples include `span`, `a`, and `strong`. This default behavior affects layout, which is why many layout bugs come from treating inline elements like block-level ones or vice versa.

Question 3: What is semantic HyperText Markup Language, and why do product companies care about it?

Answer: Semantic HyperText Markup Language means choosing tags that describe the meaning of content, not just its appearance. For example, use `nav` for navigation, `main` for primary content, and `article` for self-contained pieces. Search engines, screen readers for blind users, and other tools rely on these tags to understand page structure. Product companies prefer semantic markup because it improves accessibility, search ranking, and long-term maintainability.

Question 4: When should you use a `div` versus a `section` or `article`?

Answer: Use `section` when content is a thematic grouping that usually has a heading, such as a features area on a landing page. Use `article` when content stands alone and could be reused elsewhere, such as a blog post or product card. Use `div` only when no semantic tag fits—it is a generic container with no meaning. Overusing `div` makes pages harder for assistive technology and search engines to parse.

Question 5: What is the Document Object Model, and how does HyperText Markup Language relate to it?

Answer: The Document Object Model is a tree-shaped representation of a web page that the browser builds after reading HyperText Markup Language. Each tag becomes a node in the tree, and Cascading Style Sheets and JavaScript can read and change those nodes. When interviewers ask you to "inspect the Document Object Model," they mean use browser developer tools to see how HyperText Markup Language was parsed and styled. Understanding this tree helps you debug why a style or script affects one element but not another.

Question 6: Explain the Cascading Style Sheets box model. What are content, padding, border, and margin?

Answer: Every element is drawn as a rectangular box with four layers. Content is the inner area where text and images live. Padding is empty space inside the border, between content and the border edge. Border wraps the padding. Margin is empty space outside the border, separating this box from neighboring boxes. Total space an element occupies depends on how `box-sizing` is set, which is a common source of layout surprises.

Question 7: What does `box-sizing: border-box` do, and why is it widely used?

Answer: By default, `box-sizing: content-box` means width and height apply only to the content area; padding and border add extra size. With `box-sizing: border-box`, the declared width and height include content, padding, and border together. This makes layout math predictable: a `width: 200px` element stays 200 pixels wide even after you add padding. Many codebases set `border-box` globally because it reduces accidental overflow and broken grids.

Question 8: A button looks wider than its neighbor even though both have `width: 100px`. What box model issues could cause this?

Answer: Check whether `box-sizing` differs between elements—one might still use `content-box`. Compare padding, border width, and margin; unequal horizontal padding or a visible border adds pixels. Inline-block or flex items can also shrink or grow based on content unless you set `flex-shrink: 0` or consistent box rules. Open developer tools, inspect computed width, and verify padding, border, and box-sizing match on both buttons.

Question 9: What is Cascading Style Sheets specificity, and how is it calculated?

Answer: Specificity decides which Cascading Style Sheets rule wins when multiple rules target the same property on the same element. Inline styles beat identifiers, identifiers beat classes and attributes, and classes beat element selectors. A common mental model: count inline styles, then identifiers (`#id`), then classes (`.class`, `[attr]`, `:pseudo-class`), then element types (`div`, `p`). The more specific selector applies unless importance (`!important`) or source order breaks the tie.

Question 10: Two rules set the same property on one element. One uses a class selector, the other an element selector. Which wins?

Answer: The class selector wins because it is more specific. A class selector counts as one class-level point, while an element selector counts as one element-level point, and class-level beats element-level. If both had equal specificity, the rule that appears later in the stylesheet would win. Interviewers often follow up by asking you to avoid overusing identifiers or `!important` to force wins.

Question 11: What is the cascade in Cascading Style Sheets?

Answer: The cascade is the set of rules the browser uses to combine styles from multiple sources: user agent defaults, user settings, author stylesheets, and inline styles. It considers origin, importance, specificity, and source order. "Cascading" means styles flow together and conflicts resolve predictably rather than the last file always winning blindly. Understanding the cascade helps you explain why a component library style overrides your local class—or does not.

Question 12: What is Flexbox, and what problems does it solve?

Answer: Flexbox (Flexible Box Layout) is a one-dimensional layout system for arranging items in a row or a column. It excels at distributing space, aligning items vertically and horizontally, and handling uneven content sizes. You apply `display: flex` on a container and control direction with `flex-direction`, alignment with `align-items` and `justify-content`, and how children grow or shrink with `flex` properties. Product teams use Flexbox constantly for navigation bars, card rows, and form toolbars.

Question 13: How would you center a child both horizontally and vertically inside a parent using Flexbox?

Answer: Set the parent to `display: flex`, then use `justify-content: center` for horizontal centering along the main axis and `align-items: center` for vertical centering along the cross axis. Ensure the parent has a defined height if you need vertical centering in the full viewport area. Example:

```
.parent {  
  display: flex;  
  justify-content: center; /* horizontal along row direction */  
  align-items: center;    /* vertical along row direction */  
  min-height: 200px;  
}
```

Question 14: What is the difference between `justify-content` and `align-items` in Flexbox?

Answer: `justify-content` aligns items along the main axis—the direction set by `flex-direction` (row or column). `align-items` aligns items along the cross axis, perpendicular to the main axis. In a row layout, main axis is usually horizontal and cross axis vertical, so `justify-content` handles left-right distribution and `align-items` handles top-bottom alignment. Swapping `flex-direction` swaps which property controls which visual direction.

Question 15: What is Cascading Style Sheets Grid, and when would you choose Grid over Flexbox?

Answer: Grid is a two-dimensional layout system: you define rows and columns at once and place items into cells. Choose Grid for page-level layouts, dashboards, and photo galleries where both axes matter. Choose Flexbox for single rows or columns where items flow in one direction. Many production layouts combine them: Grid for the overall page skeleton, Flexbox for alignment inside each cell.

Question 16: How do you create a three-column layout with Grid where the center column is wider?

Answer: Define column tracks with `grid-template-columns`, giving the side columns fixed or fractional units and the center a larger fraction. Example:

```
.layout {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr; /* center twice as wide as sides */  
  gap: 16px;  
}
```

Use `gap` for consistent spacing between columns without margin hacks. On small screens, add a media query to stack columns into one track.

Question 17: Explain the five `position` values: static, relative, absolute, fixed, and sticky.

Answer: `static` is the default; the element stays in normal document flow. `relative` keeps the element in flow but offsets it with `top`, `right`, `bottom`, or `left` without moving siblings. `absolute` removes the element from flow and positions it relative to the nearest positioned ancestor. `fixed` positions relative to the viewport and stays put while scrolling. `sticky` behaves like relative until a scroll threshold, then acts like fixed within its container.

Question 18: Why does `position: absolute` sometimes appear in the wrong place?

Answer: An absolutely positioned element anchors to the nearest ancestor that is not `static`—often a forgotten `position: relative` on a parent is missing, so it jumps to the page root. Missing width constraints can make it collapse oddly. Check ancestor positioning, padding, and overflow settings. In interviews, drawing the ancestor chain and identifying the containing block shows strong debugging skill.

Question 19: What is responsive design?

Answer: Responsive design means building interfaces that adapt gracefully to different screen sizes, input methods, and orientations without separate sites for each device. Techniques include fluid widths, flexible images, relative units, media queries, and layouts that reflow instead of horizontal scrolling. The goal is usable content on phones, tablets, and large monitors from one HyperText Markup Language base. Product companies expect you to reason about breakpoints by content, not only by device models.

Question 20: What is a media query in Cascading Style Sheets?

Answer: A media query applies styles only when certain conditions are true, most often viewport width. Example:

```
@media (min-width: 768px) {  
  .sidebar { display: block; }  
}
```

This lets you change layout, typography, or visibility at breakpoints. Media queries can also check orientation, prefers-reduced-motion, and other environment features. They are the primary tool for adapting one codebase to many screen sizes.

Question 21: What is mobile-first design, and how does it differ from desktop-first?

Answer: Mobile-first means you write base styles for small screens, then use `min-width` media queries to enhance layout for larger screens. Desktop-first starts with large-screen styles and uses `max-width` queries to scale down. Mobile-first aligns with performance and progressive enhancement because phones load simpler styles first. Interviewers may ask you to refactor desktop-first code into mobile-first order.

Question 22: What are viewport units (`vw` , `vh` , `vmin` , `vmax`), and what pitfalls exist on mobile browsers?

Answer: Viewport width (`vw`) and viewport height (`vh`) are percentages of the browser viewport: `1vw` is one percent of viewport width. `vmin` and `vmax` use the smaller or larger of width and height. On mobile, dynamic browser chrome (address bars showing and hiding) can make `100vh` taller than the visible area, causing content to sit under toolbars. Newer units like `dvh` (dynamic viewport height) address this; always test full-screen sections on real phones.

Question 23: How would you make an image responsive so it never overflows its container?

Answer: Set `max-width: 100%` and `height: auto` on the image so it scales down with its container but keeps aspect ratio. Ensure the parent does not force a fixed smaller width with overflow hidden unless intentional. For art-directed crops, use `object-fit: cover` inside a sized container. HyperText Markup Language attribute `width` and `height` can still be set to reserve space and reduce layout shift.

Question 24: What is web accessibility, and why is it required at large product companies?

Answer: Web accessibility means people with disabilities—including blind, low-vision, deaf, motor, and cognitive differences—can perceive, navigate, and interact with your site. Legal

requirements and company policies often mandate conformance with Web Content Accessibility Guidelines. Accessible products reach more users, reduce support costs, and survive audits. Interviewers expect you to connect semantic HyperText Markup Language, keyboard support, and visible focus to real user outcomes.

Question 25: What is Accessible Rich Internet Applications, and when should you use it?

Answer: Accessible Rich Internet Applications is a set of attributes that extend HyperText Markup Language so assistive technologies understand custom widgets like tabs, dialogs, and comboboxes. Use native elements (`button` , `input` , `select`) first because they ship with built-in behavior and accessibility. Reach for Accessible Rich Internet Applications when no native element matches your user interface pattern, and then follow established patterns such as `role="dialog"` with `aria-modal="true"` and labelled headings.

Question 26: What is the difference between `aria-label` and `aria-labelledby` ?

Answer: `aria-label` provides an accessible name directly as an attribute string on the element. `aria-labelledby` points to one or more existing elements whose text should compose the name, similar to how a `label` element references an input. Prefer visible text referenced by `aria-labelledby` when possible so sighted and assistive technology users hear the same wording. Avoid duplicating labels in both places unless necessary.

Question 27: How do you make a custom dropdown keyboard-accessible?

Answer: Use a `button` to toggle visibility, move focus with arrow keys inside the list, close on Escape, and select with Enter or Space. Manage `aria-expanded` on the trigger and `role="listbox"` with `role="option"` items, or use a proven pattern from a design system. Ensure focus is trapped or returned sensibly and that visible focus outlines remain. Interviewers want to hear that mouse-only click handlers are insufficient.

Question 28: What is visible focus, and why should you avoid `outline: none` without a replacement?

Answer: Visible focus is a clear indicator showing which element receives keyboard input, usually a ring or border around links and buttons. Removing the default outline with `outline:`

`none` without substituting a custom `:focus-visible` style hides navigation cues for keyboard users. Product accessibility reviews flag missing focus states. A good pattern styles `:focus-visible` prominently while suppressing mouse-click rings when appropriate.

Question 29: How should you associate a text label with a form input in HyperText Markup Language?

Answer: Wrap the input inside a `label` or set the label's `for` attribute to match the input's unique `id`. Clicking the label then focuses the field, which helps motor-impaired users and increases hit area on mobile. Placeholder text alone is not a label because it disappears when typing and is often skipped by screen readers as a name. Every meaningful input should have a visible, programmatically associated label.

Question 30: What input types and attributes improve forms on mobile devices?

Answer: Use `type="email"`, `type="tel"`, and `type="number"` to show appropriate keyboards. Attributes like `autocomplete`, `inputmode`, and `required` guide browsers and assistive tools. Set `name` attributes for autofill and server handling. Large touch targets (at least 44 by 44 pixels) and clear error messages tied with `aria-describedby` reduce failed submissions. Product form interviews often include validation and accessibility together.

Question 31: What is Block Element Modifier methodology in Cascading Style Sheets?

Answer: Block Element Modifier is a naming convention for classes: Block is the component (`card`), Element is a part (`card__title`), and Modifier is a variant (`card--featured`). Double underscores separate block and element; double hyphens mark modifiers. It avoids deep selector chains and keeps styles reusable and searchable. Teams at scale use it to prevent global class name collisions and to document component application programming interface in class names.

Question 32: Write Block Element Modifier class names for a primary button and its disabled state.

Answer: Block: `button`. Element examples: `button__icon`, `button__label`. Modifier for primary style: `button--primary`. Modifier for disabled state: `button--disabled`. HyperText

Markup Language might look like `<button class="button button--primary">Pay now</button>`. Cascading Style Sheets targets `.button`, `.button--primary`, and `.button--disabled` separately instead of nesting many selectors.

Question 33: What are Cascading Style Sheets custom properties (often called variables), and how do they differ from preprocessor variables?

Answer: Cascading Style Sheets custom properties are declared with `--token-name: value` and used with `var(--token-name)`. They live in the cascade, inherit, and can change at runtime or inside media queries. Preprocessor variables from tools like Sass compile away before the browser runs and cannot respond to user themes dynamically. Custom properties enable theming, dark mode, and design tokens shared across components.

Question 34: How would you implement light and dark themes using Cascading Style Sheets custom properties?

Answer: Define color tokens on `:root` for the default theme, then override them inside a selector such as `[data-theme="dark"]` or `@media (prefers-color-scheme: dark)`. Components consume `background: var(--surface)` instead of hard-coded hex values. Toggling theme becomes switching an attribute on `html` or `body`. This pattern scales well in design systems at companies like Salesforce and Stripe.

Question 35: What are transforms in Cascading Style Sheets, and do they trigger layout recalculation?

Answer: Transforms such as `translate`, `scale`, `rotate`, and `skew` visually move or reshape an element without changing its layout box in the document flow. They typically run on the compositor thread and are cheaper than changing `top` or `left` for animations. `transform: translateX(100px)` slides visually; layout neighbors stay put. Overusing transforms for readable text positioning can still harm accessibility if focus order does not match visual order.

Question 36: What is the difference between a transition and an animation in Cascading Style Sheets?

Answer: A transition smoothly changes property values when a trigger occurs, such as hover changing `opacity` over 200 milliseconds with `transition: opacity 200ms ease`. An

animation uses `@keyframes` to define multiple steps over time with `animation-name`, duration, and iteration. Use transitions for simple state changes; use animations for complex motion. Respect `prefers-reduced-motion: reduce` to turn off or shorten motion for users who request it.

Question 37: Explain z-index and stacking contexts. Why does a high z-index sometimes fail?

Answer: `z-index` controls paint order within the same stacking context; higher values appear in front. A new stacking context is created by properties like `position` with z-index, `opacity` less than 1, `transform`, and `filter`. A child with `z-index: 9999` cannot escape above a sibling context of its parent. Fix by raising the parent's z-index or restructuring contexts, not only cranking the child's number.

Question 38: A modal dialog appears behind the header even though the modal has a larger z-index. How do you debug?

Answer: Compare stacking contexts: the header may sit in a context created by `transform` or `isolation: isolate` with a higher parent z-index. Inspect ancestors of both modal and header in developer tools and note who creates contexts. Often the fix is giving the modal wrapper `position: fixed` and a z-index higher than the header's context, or lowering the header's context when modals open. Document Object Model order alone does not fix z-index across contexts.

Question 39: What is critical Cascading Style Sheets, and why does it matter for performance?

Answer: Critical Cascading Style Sheets is the minimal set of styles needed to render above-the-fold content on first paint. Inlining critical styles in the document head avoids waiting for full stylesheets before users see layout. Non-critical styles load asynchronously. Product companies measure Largest Contentful Paint and Cumulative Layout Shift; slow or render-blocking styles hurt those metrics and search ranking.

Question 40: How does lazy loading images improve performance, and what HyperText Markup Language attribute enables it?

Answer: Lazy loading defers downloading off-screen images until the user scrolls near them, saving bandwidth and main-thread work on initial load. Native lazy loading uses `loading="lazy"` on `img` and `iframe` elements in supporting browsers. Always set `width` and `height` or aspect-ratio to prevent layout shift when images arrive. For hero images needed immediately, do not lazy load them.

Question 41: What is render-blocking resources, and how do stylesheets block rendering?

Answer: Render-blocking resources prevent the browser from painting until they are processed. By default, linked Cascading Style Sheets in the head block rendering because the browser wants to avoid flashing unstyled content. JavaScript without `defer` or `async` can also block parsing. Mitigations include critical Cascading Style Sheets, `media` attributes, preload for fonts, and moving non-essential scripts to the end or deferring them.

Question 42: What is Cumulative Layout Shift, and which HyperText Markup Language and Cascading Style Sheets habits reduce it?

Answer: Cumulative Layout Shift measures unexpected movement of content while the page loads—such as text jumping when an image or ad appears. Reduce it by reserving space for images and embeds, avoiding inserting content above existing content unless user-initiated, using `font-display: swap` carefully with matched fallback metrics, and not animating layout-affecting properties like `height` without user action. Product teams track this as a Core Web Vital.

Question 43: How would you build a sticky site header that stays visible on scroll?

Answer: Apply `position: sticky; top: 0; z-index: 10` to the header element. Ensure no ancestor has `overflow: hidden` that breaks sticky behavior. Give the header a background so content does not show through while scrolling. Test on mobile browsers where sticky inside transformed parents fails. Pair with sufficient contrast and skip links so keyboard users can bypass repetitive navigation.

Question 44: Implement a simple responsive navigation: horizontal on desktop, hamburger menu on mobile (describe approach).

Answer: Markup: semantic `nav` with a list of links and a `button` toggling a mobile panel. Base mobile-first styles stack links vertically and hide them behind the toggle using classes or `hidden` attribute with Accessible Rich Internet Applications `aria-expanded`. At `min-width: 768px`, use a media query to show links in a row with Flexbox and hide the menu button. JavaScript only toggles state; layout lives in Cascading Style Sheets. Focus trap in the mobile panel is a strong follow-up answer.

Question 45: What is the difference between `visibility: hidden` and `display: none`?

Answer: `display: none` removes the element from layout entirely—it takes no space and is ignored by screen readers in most cases. `visibility: hidden` hides visually but the element still occupies space in layout. For accessible hiding that removes content from everyone including assistive tech, prefer `hidden` attribute or `display: none`. For visibility toggles in animations, `visibility` paired with `opacity` is common.

Question 46: What are pseudo-classes and pseudo-elements? Give examples of each.

Answer: Pseudo-classes select elements in a particular state, such as `:hover`, `:focus-visible`, and `:nth-child(2)`. Pseudo-elements style specific parts of an element, such as `::before`, `::after`, and `::placeholder`. Pseudo-classes use a single colon; pseudo-elements use double colon in modern Cascading Style Sheets. Misusing `::before` for essential content hurts accessibility because generated content may not be in the accessibility tree reliably.

Question 47: How do you prevent horizontal overflow on mobile layouts?

Answer: Audit elements with fixed widths wider than the viewport, large negative margins, and 100 viewport width sections that ignore scrollbar width. Use `max-width: 100%` on media, `overflow-x: clip` or `hidden` on `body` only after fixing root causes, and prefer `%` or `fr` units in Grid and Flexbox. Developer tools device mode helps find the offending box by toggling `overflow` on suspects.

Question 48: What is the purpose of the `alt` attribute on images, and when can it be empty?

Answer: The `alt` attribute provides a text alternative when the image cannot be seen, read by screen readers and shown when the image fails to load. Decorative images that convey no information should use `alt=""` so assistive technology skips them. Informative images need concise descriptions of content or function, not filenames. Functional images such as icons in buttons should describe the action, not the picture.

Question 49: A interviewer asks: "Your flex item is overflowing and not shrinking. What properties do you check?"

Answer: Inspect `flex-shrink` —default is 1, but `flex-shrink: 0` prevents shrinking. Check `min-width: auto` on flex items, which defaults to content minimum and can block shrink; setting `min-width: 0` often fixes overflow in flex rows. Long unbreakable text may need `overflow-wrap: break-word`. Also verify parent width constraints and whether `flex-basis` forces a large starting size.

Question 50: What HyperText Markup Language and Cascading Style Sheets practices would you follow before shipping a feature at a product company?

Answer: Use semantic HyperText Markup Language and test with keyboard-only navigation and a screen reader sample pass. Keep selectors maintainable (Block Element Modifier or design tokens), avoid magic numbers without comments, and verify responsive breakpoints on real devices. Measure performance impact: lazy load non-critical images, minimize render-blocking styles, and watch layout shift. Run automated accessibility linting, cross-browser smoke tests, and document any Accessible Rich Internet Applications widgets you introduce so the next engineer can extend them safely.